

DOSSIER D'ARCHITECTURE LOGICIELLE

Transition IAM, Gestion du Legacy et Dette Technique

1. Contexte : Du Monolithe à l'Architecture Fédérée

L'un des défis majeurs de la migration vers l'Architecture V2 a été le transfert de la responsabilité sécuritaire de l'application locale vers le système d'Identity and Access Management (IAM) d'entreprise : **Keycloak**.

Ce document détaille la transition du modèle de données (Users, Roles, Permissions) et justifie les choix architecturaux concernant le code "Legacy" actuellement maintenu dans le Backend Java.

2. Mutation du Modèle de Données (Avant / Après)



Architecture V1 (Monolithe)

Le backend Java gérait entièrement la sécurité. La base de données métier contenait :

- Table users (avec mots de passe hashés)
- Table roles
- Table permissions
- Table de jointure role_permissions



Architecture V2 Cible (IAM)

Keycloak prend le relais exclusif :

- **Suppression** totale des tables roles et permissions.
- La table users est convertie en **Proxy (Miroir)** : elle perd sa colonne *password* et son ID devient un UUID lié à Keycloak.

3. Le Modèle "User Proxy" (La Jointure Métier)

Pourquoi la table users ne disparaît-elle pas totalement ?

Bien que Keycloak gère l'authentification, le moteur métier (Spring Boot / PostgreSQL) a besoin d'établir des relations (Foreign Keys) pour savoir "À qui appartient l'incident N°45 ?". Il est impossible de faire une jointure SQL directement avec la base de données interne de Keycloak.

La solution architecturale est la création d'une table miroir réduite à sa plus simple expression :

```
@Entity
@Table(name = "users")
public class UserProxy {
    @Id
    private String id; // UUID venant de Keycloak

    private String email; // Uniquement pour l'affichage Frontend
}
```

4. Justification de la Dette Technique (Le Code Actuel)

Observation : Le fichier `User.java` actuel contient toujours les attributs `password` et `Role`, et les entités `Role.java` et `Permission.java` sont toujours présentes dans le code source.

L'Argumentaire Architectural : Dans le cadre du cycle de vie du logiciel (SDLC), une refonte ne casse jamais l'existant de manière abrupte.

Ce code est aujourd'hui classifié comme **Dette Technique / Code Legacy**. La priorité absolue de la Release V2 était de sécuriser le périmètre externe (Frontend et Endpoints API) en branchant le SSO Keycloak. Les attributs locaux de mot de passe et de rôles sont de fait **obsolètes** et ignorés par le nouveau flux de sécurité (OAuth2/JWT).

La prochaine étape d'ingénierie (Next Release) consistera à effectuer un nettoyage (Refactoring) du code mort, à exécuter des scripts de migration de données (Liquibase/Flyway) pour purger les anciennes tables de rôles, et à basculer définitivement sur l'entité `UserProxy`.